



南开大学
Nankai University



天津市智能机器人技术重点实验室
Tianjin Key Laboratory of Intelligent Robotics
南开大学机器人与信息自动化研究所
Institute of Robotics & Automatic Information System

第五章 移动机器人感知技术5

蒙特卡洛定位算法(MCL)

孙雷 教授

Tel:13512967601

Email: sunl@nankai.edu.cn

2026年5月11日

本章内容



1. 蒙特卡洛方法基本思想



2. 蒙特卡洛定位算法(MCL)



3. 自适应蒙特卡洛定位算法



4. 总结

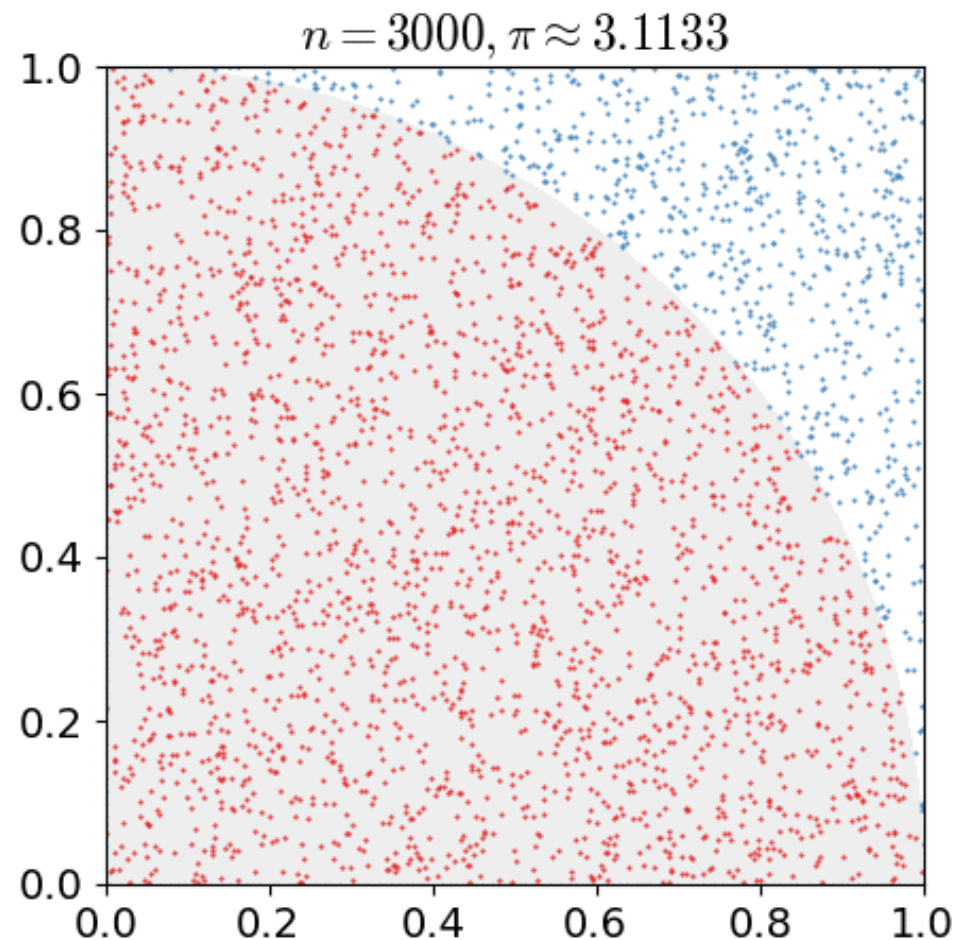
机器人定位问题分类

- 全局定位
- 局部定位（位置跟随、里程计）
 - ✓ ICP算法：
 - 局部收敛，易陷于局部最优，匹配错误
 - ✓ 收敛性受卡尔曼滤波：
 - 初始状态精度制约
- 机器人绑架问题
- 定位错误判断与恢复



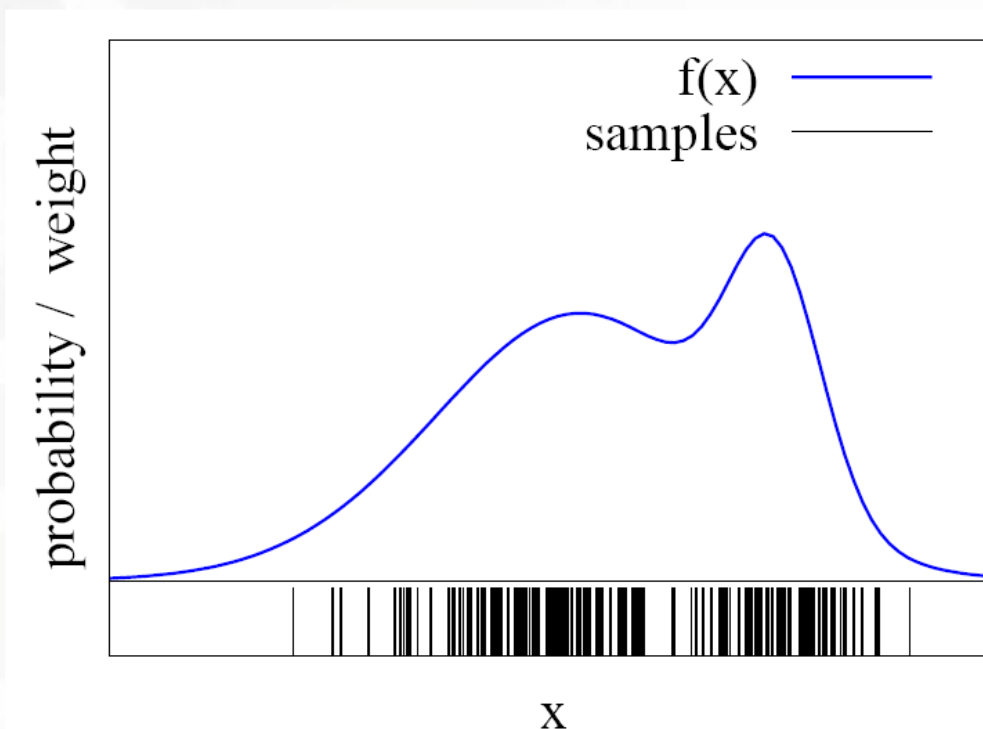
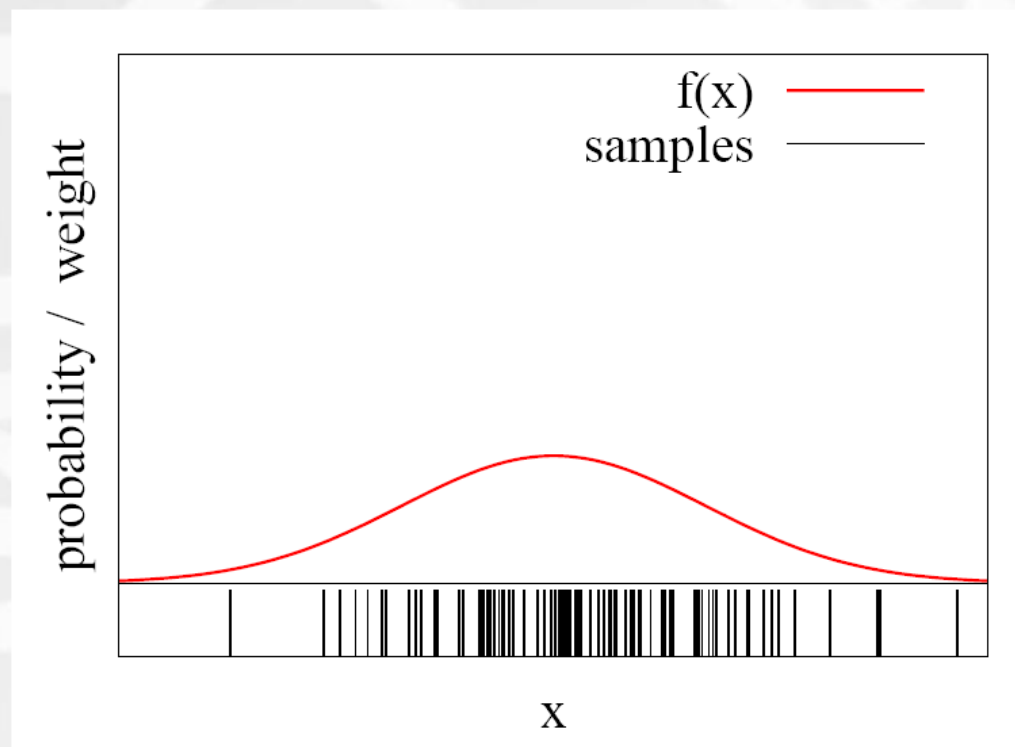
1.1 何为蒙特卡洛方法(Monte Carlo Method)

- 冯-诺伊曼（博弈论之父，原子弹设计者之一）、乌拉姆等人提出
- 一类基于概率的方法的统称
- 工作原理：
 - 不断抽样
 - 逐渐逼近
- 优点：
 - 非常通用，可以解决十分复杂的问题
- 缺点：
 - 收敛速度慢
 - 不精确
- 蒙特卡洛搜索树——AlphaGo



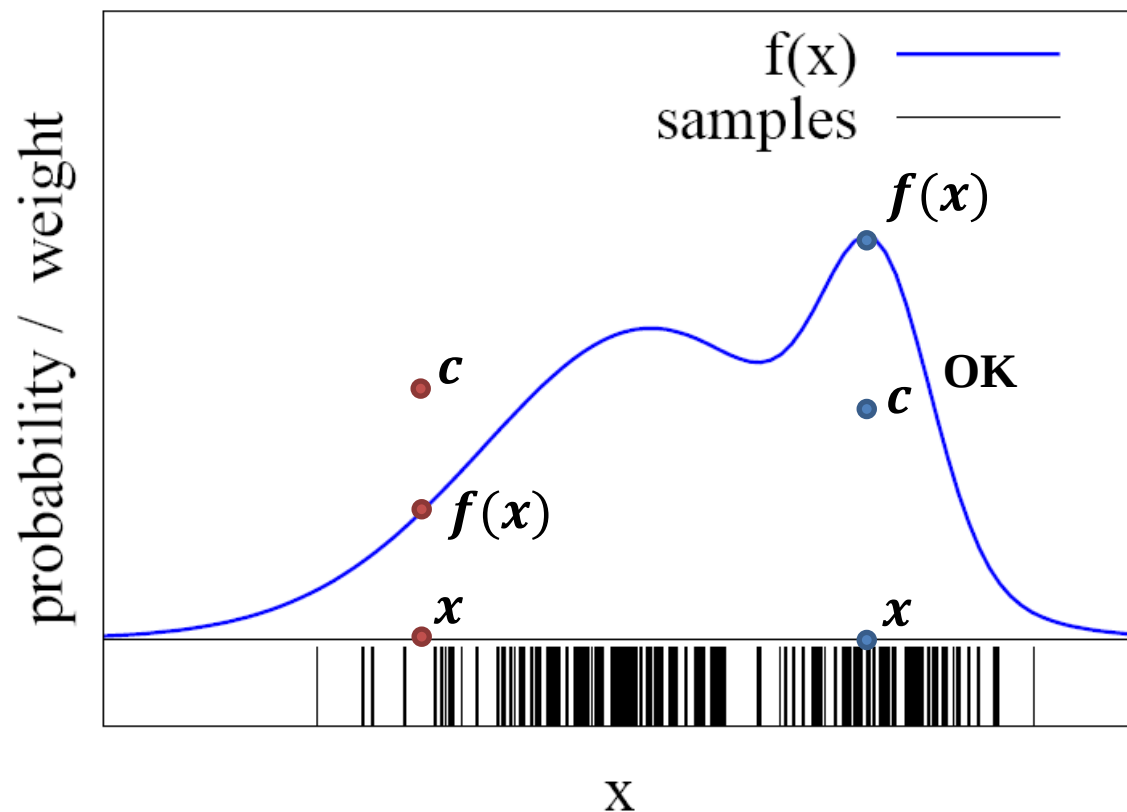
1.2 分布函数拟合

- 任意概率分布都可以通过样本 (粒子: Particle) 分布来描述
- 某一区间样本越多, 则对应的发生概率越高
- 可有效处理非高斯分布以及各种复杂分布



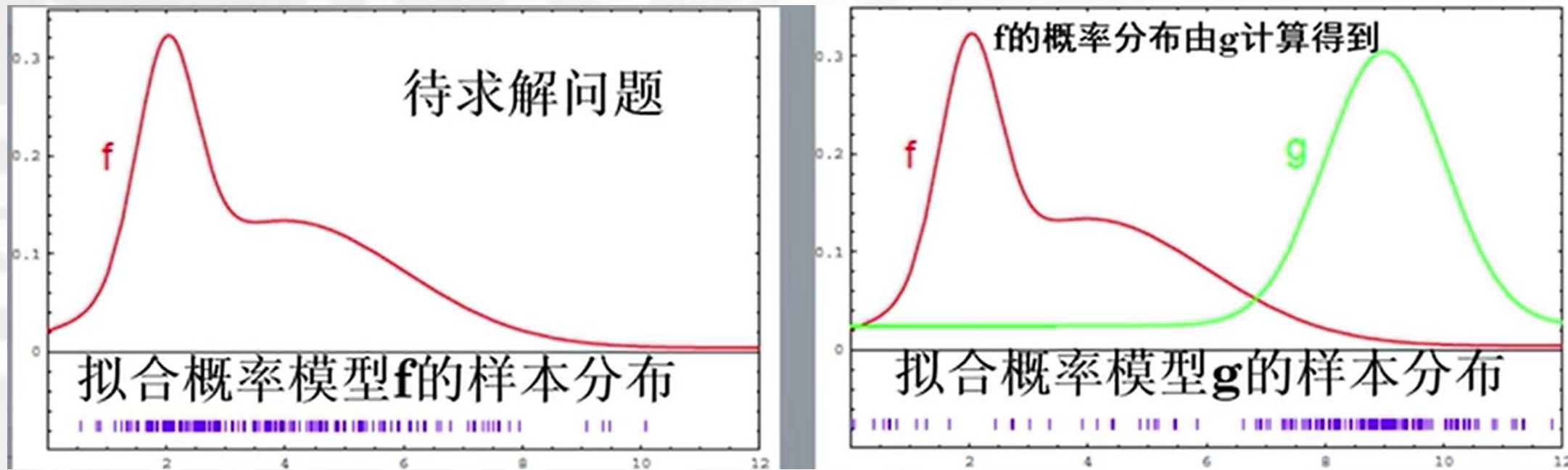
1.3 已知概率分布求取粒子分布——采样排除法

- Let us assume that $f(x) < 1$ for all x
- Sample x from a uniform distribution
- Sample c from $[0,1]$
- if $f(x) > c$ keep the sample
otherwise reject the sample



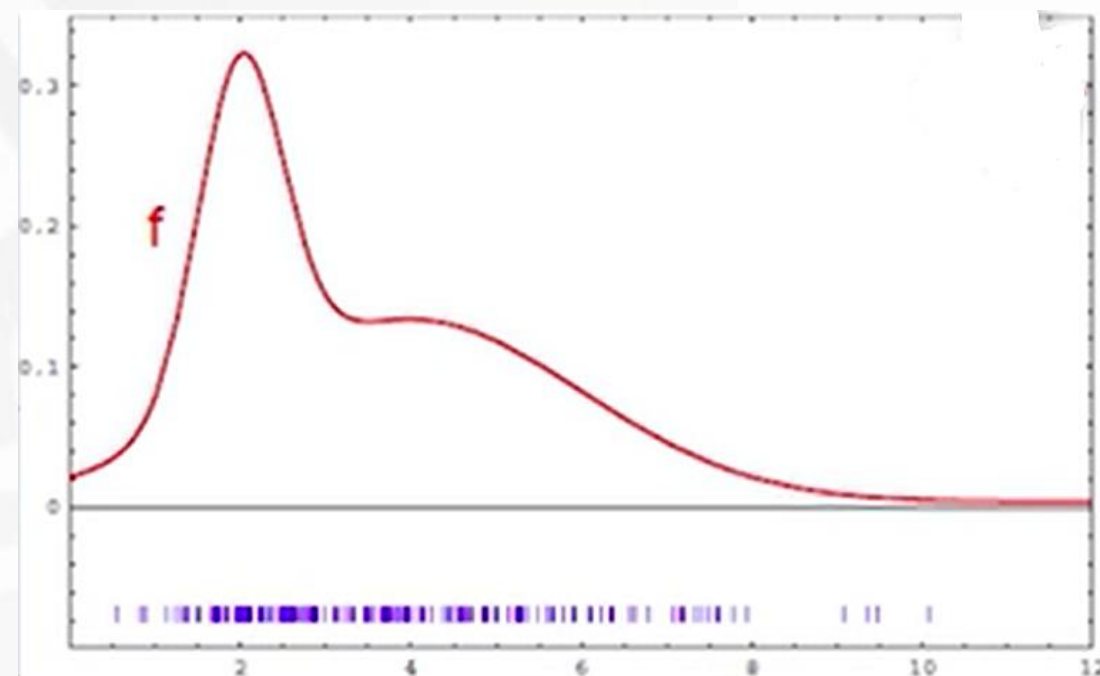
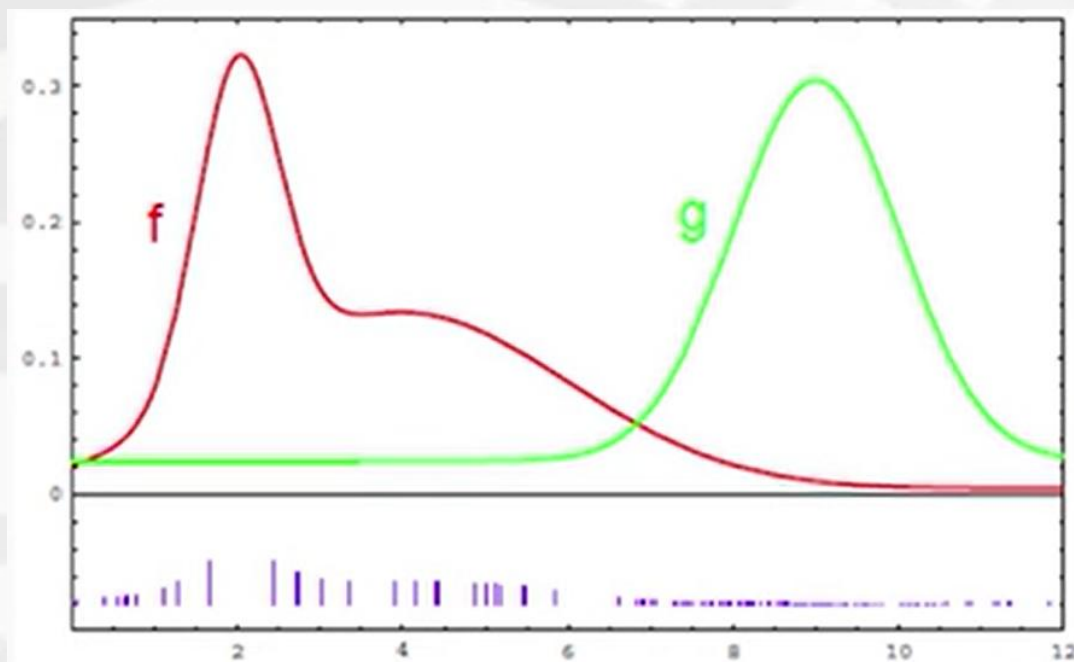
1.4 未知概率分布求取粒子分布——重要性评估

- We can even use a different distribution g (proposal distribution) to generate samples from f (target distribution)
- By introducing an importance weight w , we can account for the "differences between g and f "
- $w = f/g$



1.4 未知概率分布求取粒子分布——基于重要性的重采样原理

- 重要性指标: $\omega = f/g$ 更新概率分布
- 重采样: 利用更新后的概率分布进行重采样



均匀分布采样 g

$$\bar{\chi} = \{\langle \bar{x}^g, \bar{\omega}^g \rangle\}, \bar{\omega}^g = \frac{1}{N}$$

重要性评估更新 $\bar{\omega}^{fg}$

$$\bar{\omega}_i^{fg} = \frac{f_i}{g_i}$$

权重归一化 $\bar{\omega}^m$

$$\bar{\omega}_i^m = \frac{\bar{\omega}_i^{fg}}{\sum_{i=1}^N \bar{\omega}_i^{fg}}$$

根据重要度指标重采样

$$\bar{\chi} = \{\langle \bar{x}^f, \bar{\omega}^f \rangle\}, \bar{\omega}^f = \frac{1}{N}$$

02

蒙特卡洛定位算法

MCL: Monte Carlo Localization

2.1 蒙特卡洛定位算法（粒子滤波算法）

- Given a map of the environment, the goal of the algorithm is for the robot to determine its pose within the environment.
- At every time t , the algorithm takes as input the previous belief $X_{t-1} = \{x_{t-1}^{[1]}, x_{t-1}^{[2]}, \dots, x_{t-1}^{[M]}\}$, an actuation command u_t , and data received from sensors z_t and the algorithm outputs the new belief X_t :

$$x_t^{[i]} \sim p(x_t | z_t, u_{t-1})$$

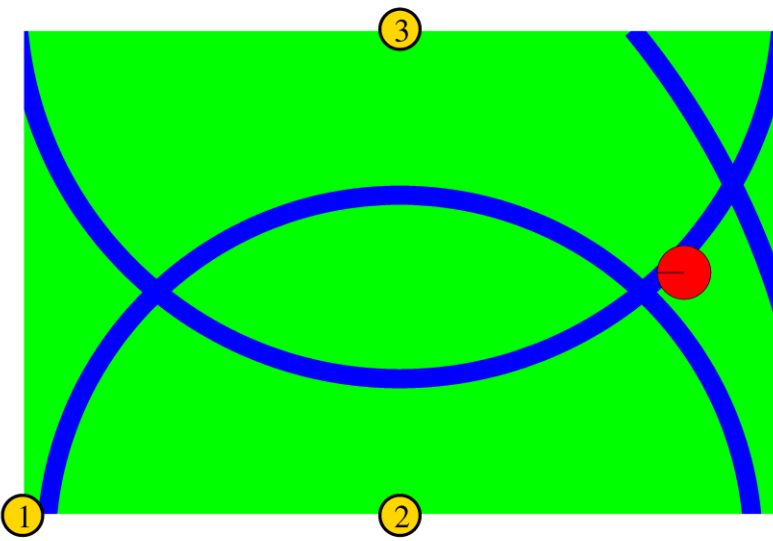
- 机器人移动路径的假设:

$$x_{0:t}^{[i]} = x_0^{[i]}, x_1^{[i]}, \dots, x_t^{[i]}$$

2.2 Example1: 2D localization



2.2.1 Distributions



Target distribution

$$p(x|z_1, z_2, z_3)$$



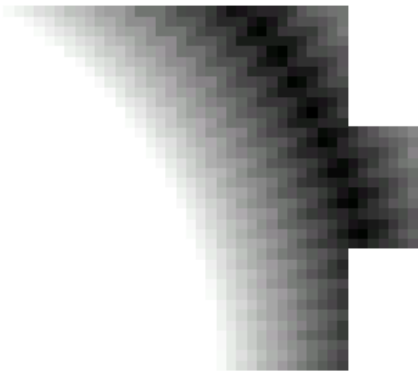
Wanted:

*samples distributed according
to Target distribution*

$$p(x|z_1, z_2, z_3)$$

Proposal distribution

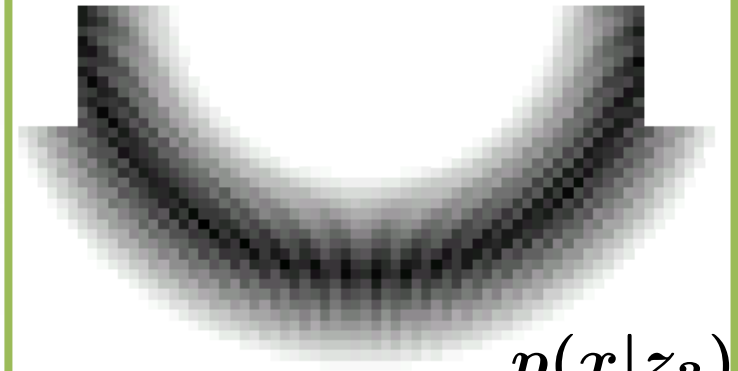
$$p(x|z_1)$$



$$p(x|z_2)$$

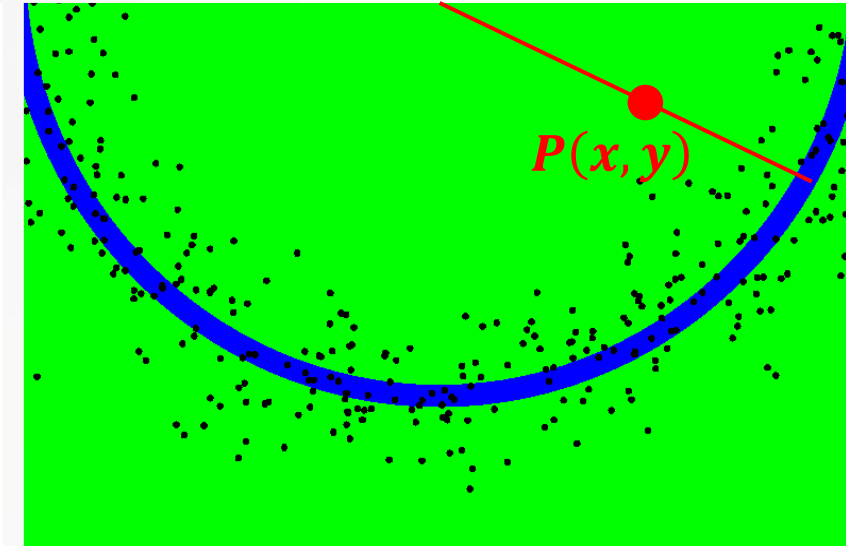
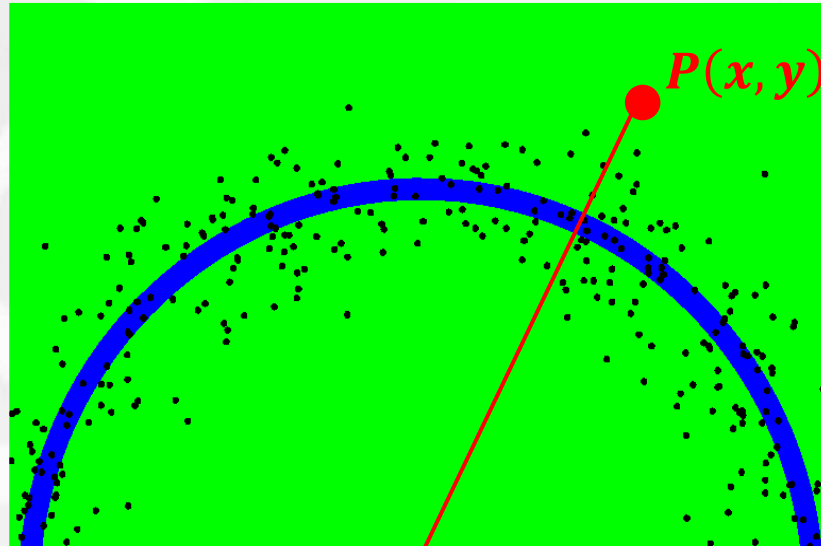
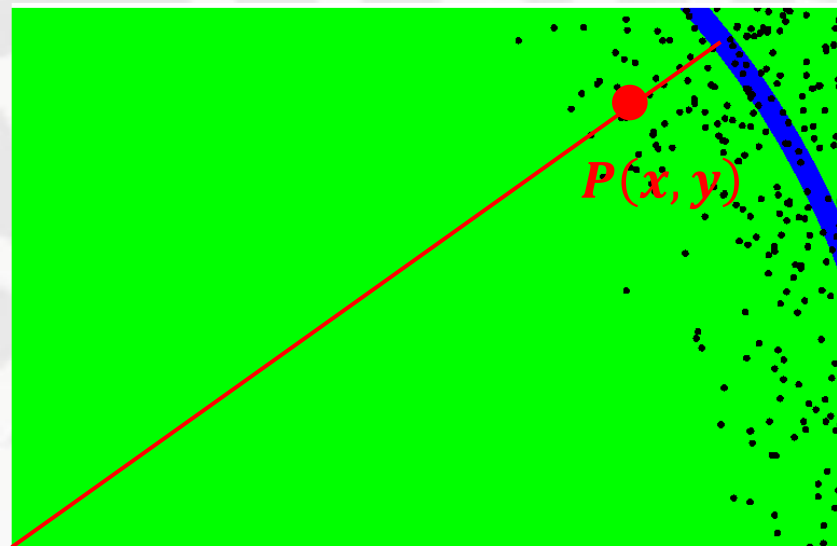


$$p(x|z_3)$$

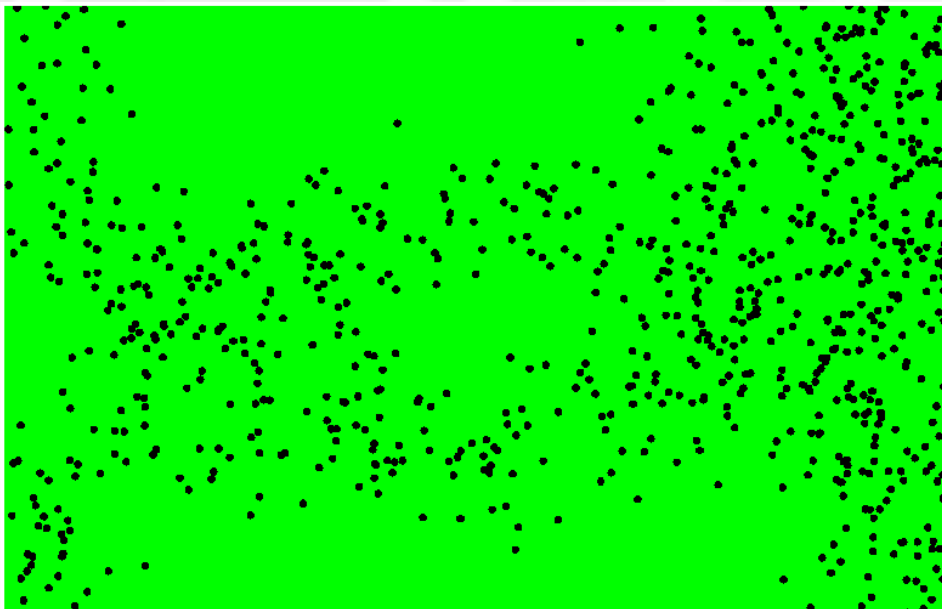


2.2.2 Proposal Distributions

(x_3, y_3)

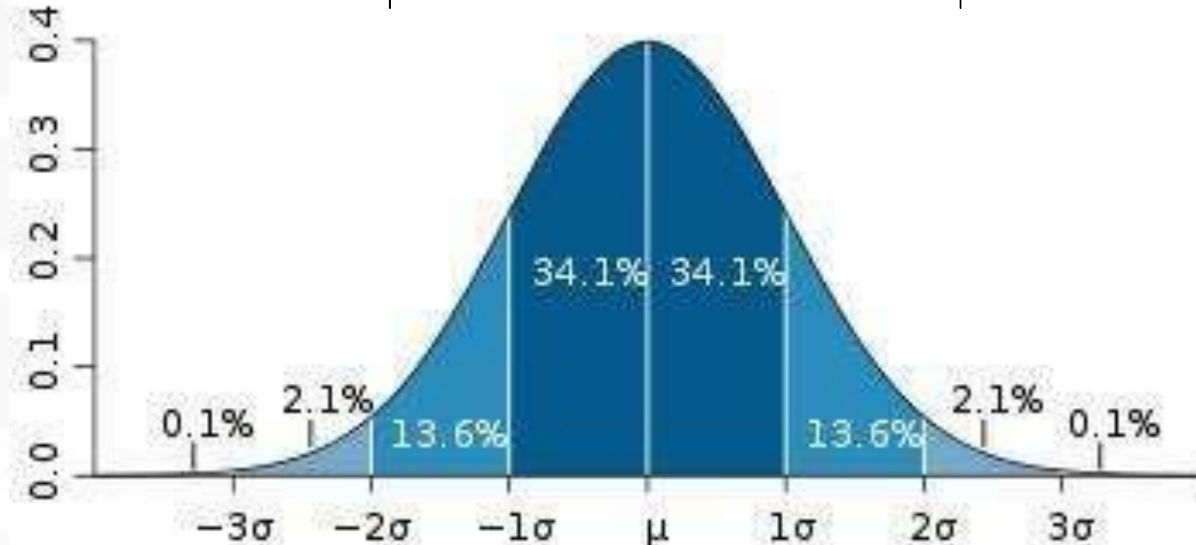


(x_1, y_1)



(x_2, y_2)

$$e_i = \left| d_i - \sqrt{(x - x_i)^2 + (y - y_i)^2} \right|$$



2.2.3 重要性评估及重采样

■ Target distribution: $f_i = [p(x_i|z_1, z_2, z_3)]$

■ 贝叶斯定理:

$$p(x_i|z_1, z_2, \dots, z_n) = \frac{\prod_k p(z_k|x_i)p(x_i)}{p(z_1, z_2, \dots, z_n)}$$

■ Proposal distribution: $g_i = [p(x_i|z_i)]$

$$p(x_i|z_i) = \frac{p(z_i|x_i)p(x_i)}{p(z_i)}$$

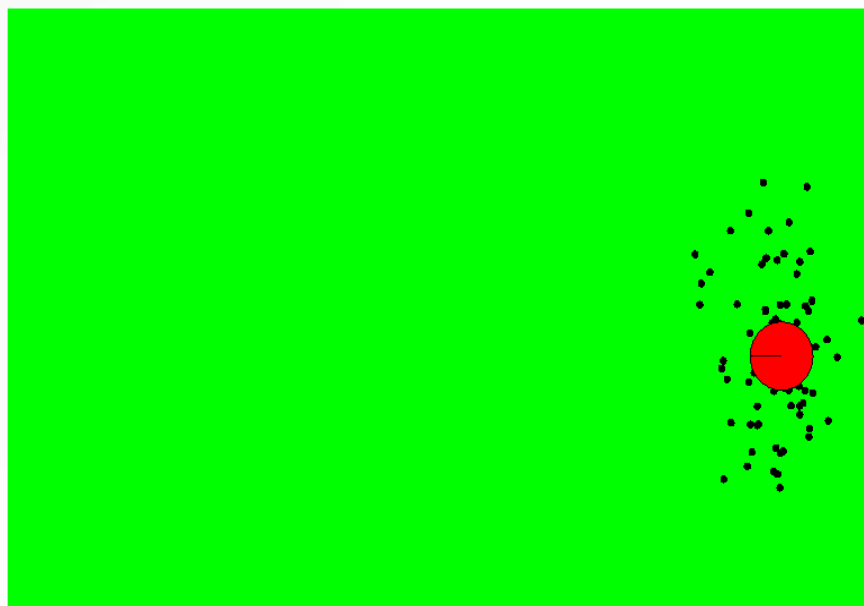
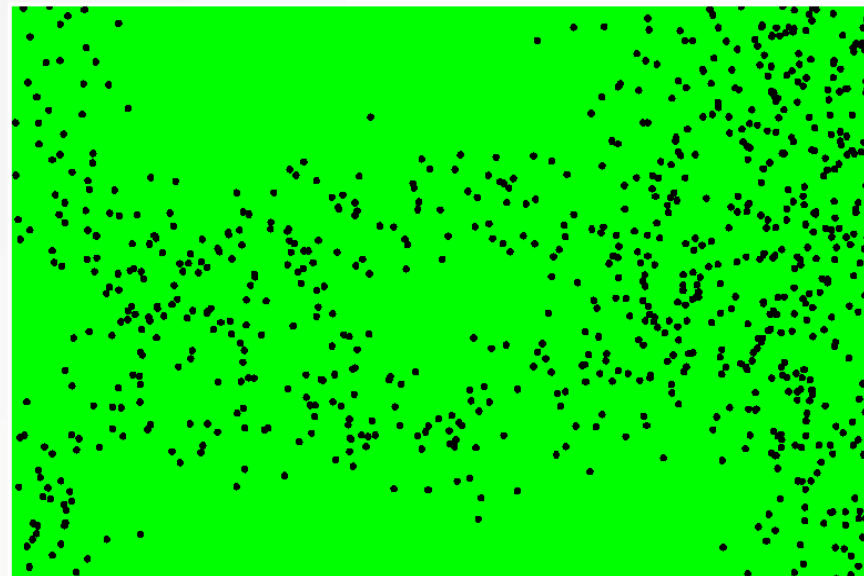
■ Importance weights w_i :

$$w_i = \frac{f_i}{g_i} = \frac{p(x_i|z_1, z_2, \dots, z_n)}{p(x_i|z_i)} = \frac{p(z_i) \prod_{k \neq i} p(z_k|x_i)}{p(z_1, z_2, \dots, z_n)}$$

■ 令

$$\eta = \frac{p(z_i)}{p(z_1, z_2, \dots, z_n)} \approx \text{Const.} \Rightarrow w_i \propto \prod_{k \neq i} p(z_k|x_i)$$

$k \neq i$: 在当前样本 (粒子) 下观测除 z_i 外其他特征的可能性



2.3 蒙特卡洛定位算法（粒子滤波算法） 流程

- Begin with the distribution of X_{t-1}
- Update and Simple based on the motion model (Proposal distribution)——prediction step
- Calculate the weight by sensor model——correction step
- Resample based on normalized weight
- Obtain the the distribution of X_t

Algorithm MCL(X_{t-1}, u_t, z_t):

$\bar{X}_t = X_t = \emptyset$

for $m = 1$ to M :

$x_t^{[m]} = \text{motion_update}(u_t, x_{t-1}^{[m]})$

$w_t^{[m]} = \text{sensor_update}(z_t, x_t^{[m]})$

$\bar{X}_t = \bar{X}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$

endfor

for $m = 1$ to M :

draw $x_t^{[m]}$ from $\bar{X}_t$ with probability $\propto w_t^{[m]}$

$X_t = X_t + x_t^{[m]}$

endfor

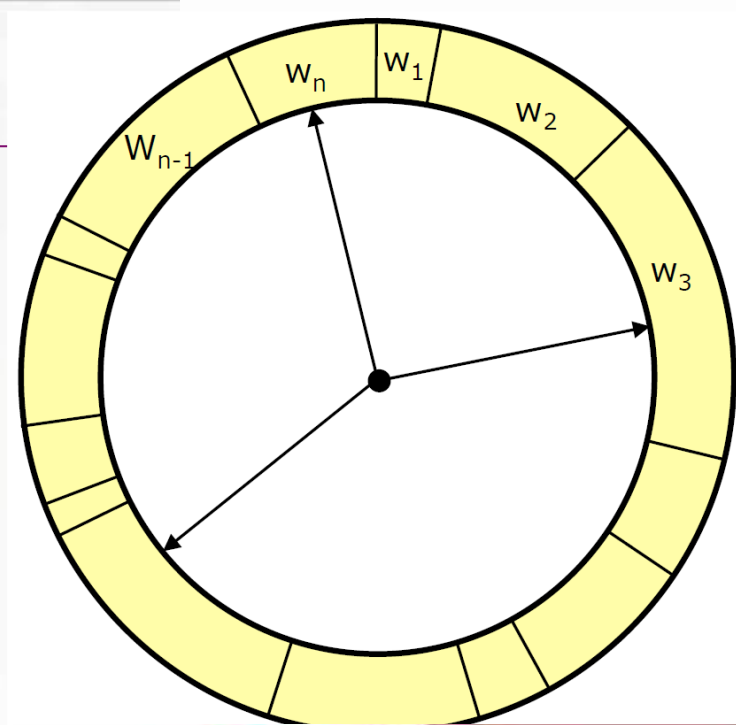
return X_t

重要性评估公式就是观测模型在样本处得到观测的可能性

2.4 重采样

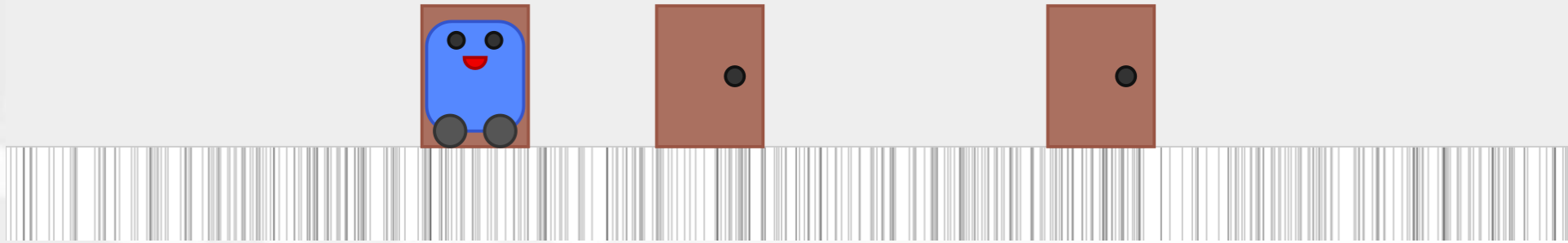
幸运转盘：

- 根据权重大小分配面积大小
- 每次抽奖Systematic resampling
- 面积越大，被抽到的次数(概率)越高
- 重采样后，点集的分布发生了变化，更加集中于权重高的部分
- 重采样后的点集，权重相同，分布密度不同



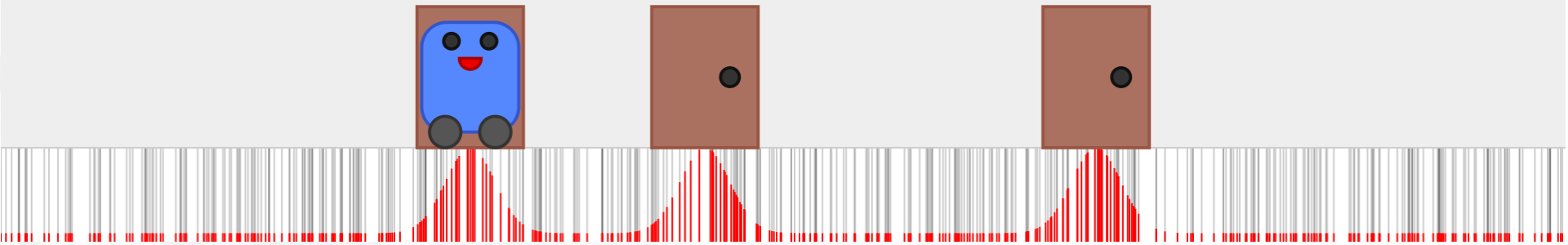
2.5 1D localization—First Step: $t = 0$

The algorithm initializes with a uniform distribution of particles.

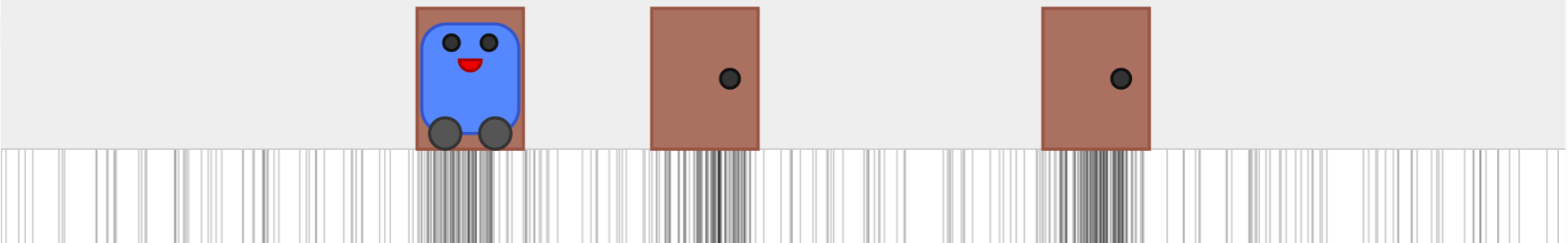


Sensor update: the robot detects a **door**. It assigns a weight to each of the particles.

$$p(z|x)$$

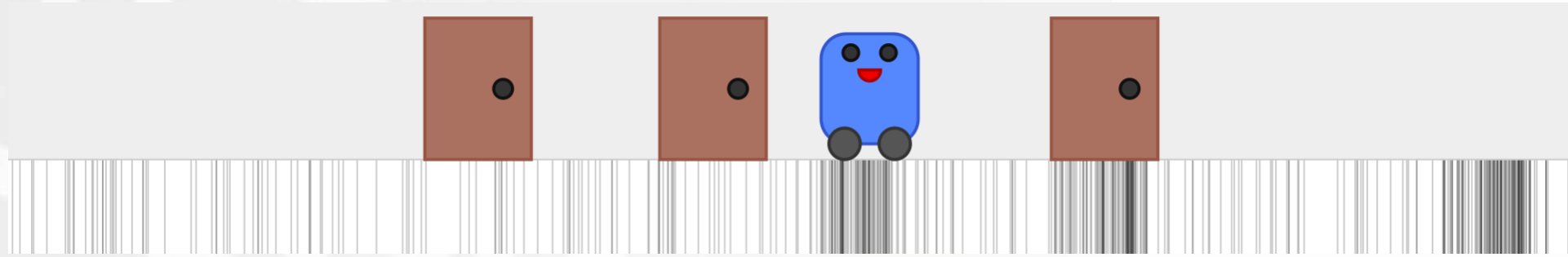


Resampling: the robot generates a set of new particles, with most of them generated around the previous particles with more weight.



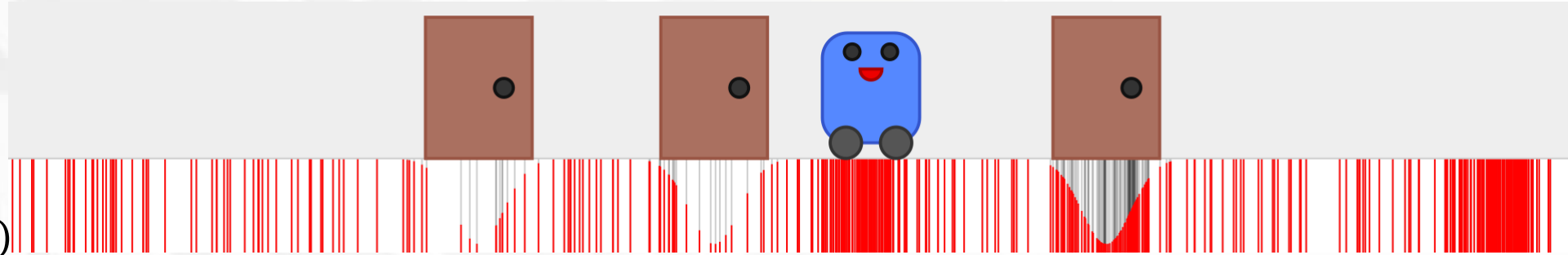
2.5 1D localization—Second Step: $t = 1$

Motion update: the robot moves some distance to the right. All particles also move right, and some noise is applied.

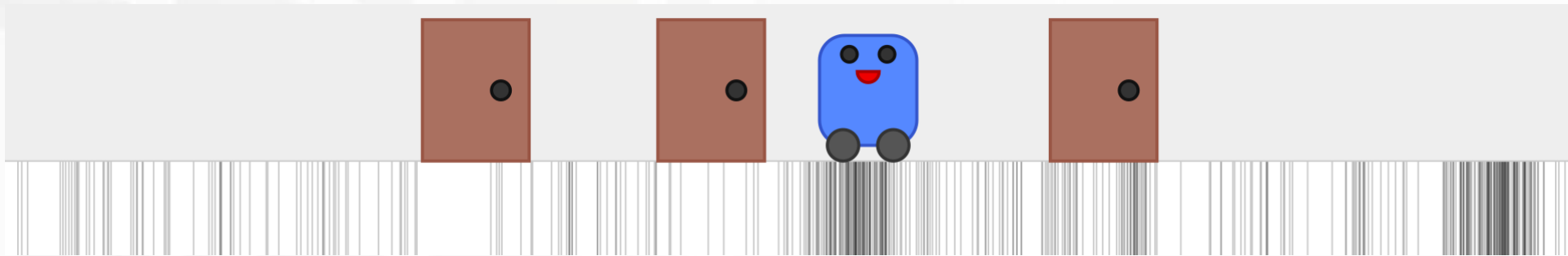


Sensor update: the robot detects **no door**. It assigns a weight to each of the particles.

$p(z|x)$

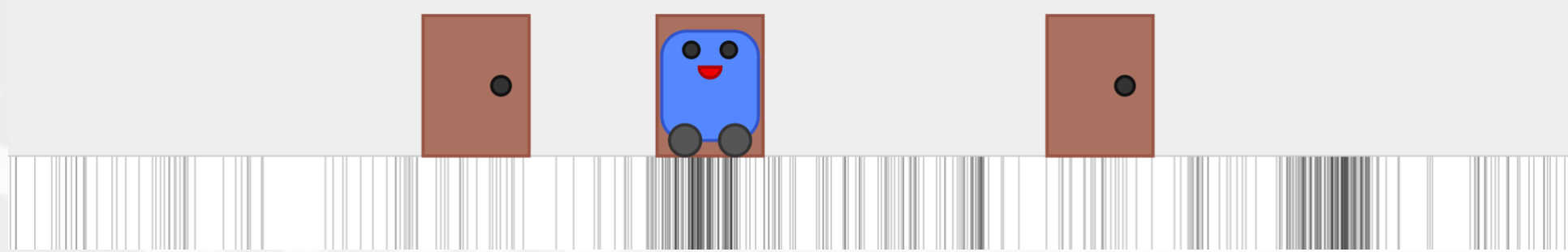


Resampling: the robot generates a set of new particles, with most of them generated around the previous particles with more weight.



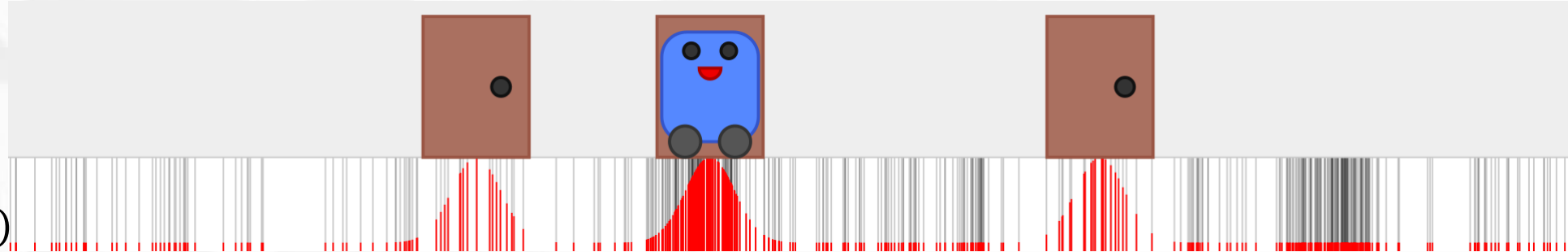
2.5 1D localization—Second Step: $t = 2$

Motion update: the robot moves some distance to the left. All particles also move left, and some noise is applied.

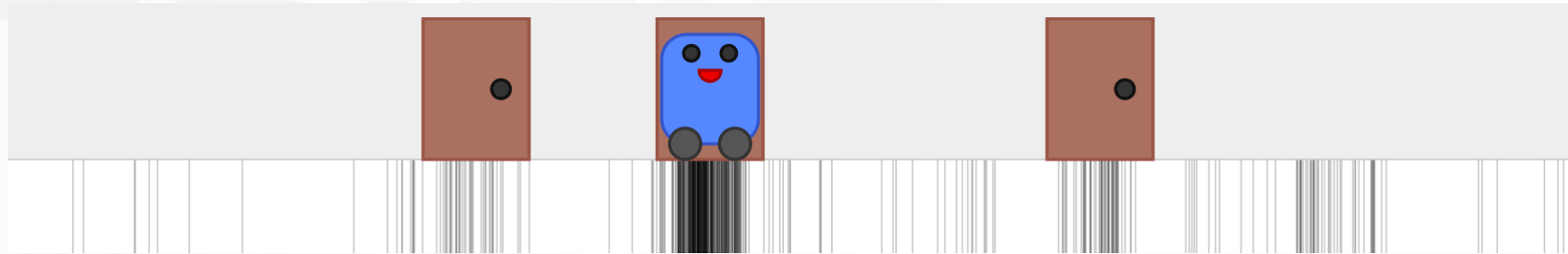


Sensor update: the robot detects a **door**. It assigns a weight to each of the particles.

$p(z|x)$



Resampling: the robot generates a set of new particles, with most of them generated around the previous particles with more weight.



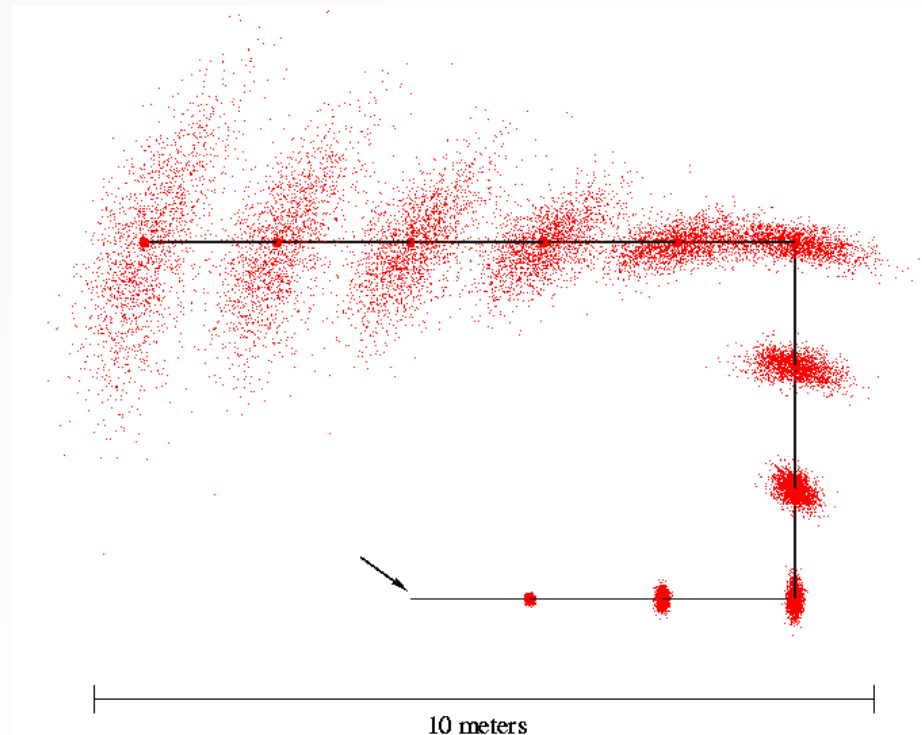
2.6 Motion Model of Mobile Robot

Error Propagation Model for Odometry

$$\Sigma_{p'} = \nabla_p f \cdot \Sigma_p \cdot \nabla_p f^\top + \nabla_{rl} f \cdot \Sigma_p \cdot \nabla_{rl} f^\top$$

二阶Runge-Kutta法:

$$\begin{cases} x_{k+1} &= x_k + v_k T_s \cos \left(\theta_k + \frac{\omega_k T_s}{2} \right) \\ y_{k+1} &= y_k + v_k T_s \sin \left(\theta_k + \frac{\omega_k T_s}{2} \right) \\ \theta_{k+1} &= \theta_k + \omega_k T_s \end{cases}$$



2.8 激光雷达观测误差模型——物理模型

■ 激光雷达传感器测量误差 $p_{hit}(z_t|x_t, m)$

- ✓ 包含随机噪声，一般服从高斯分布

■ 没有检测到障碍物 $p_{max}(z_t|x_t, m)$

- ✓ 返回激光雷达最大测距距离

■ 随机错误 $p_{rand}(z_t|x_t, m)$

- ✓ 返回错误的距离值

■ 激光雷达观测误差模型 $p(z_t|x_t, m)$:

$$p(z_t|x_t, m) = \alpha_{hit}p_{hit} + \alpha_{max}p_{max} + \alpha_{rand}p_{rand}$$

$$\alpha_{hit} + \alpha_{max} + \alpha_{rand} = 1$$

$$p(z_t|x_t, m) = \prod_k p(z_t^k|x_t, m) \text{ (其中 } k \text{ 为激光束的索引)}$$

2.8 激光雷达观测误差模型——物理模型

■ 激光雷达传感器测量误差 $p_{hit}(z_t|x_t, m)$

以真实距离为均值的高斯模型

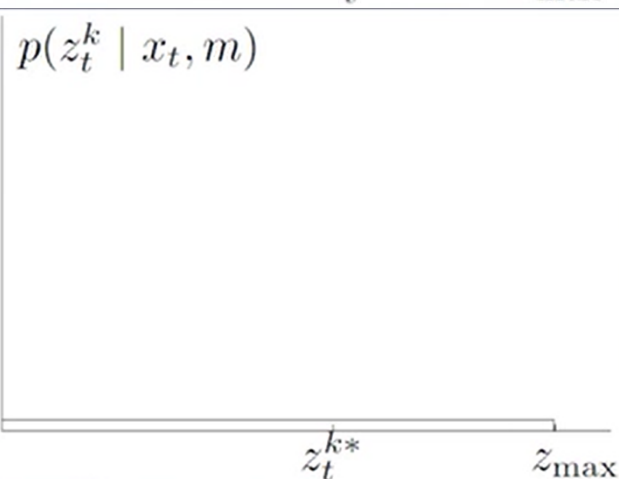
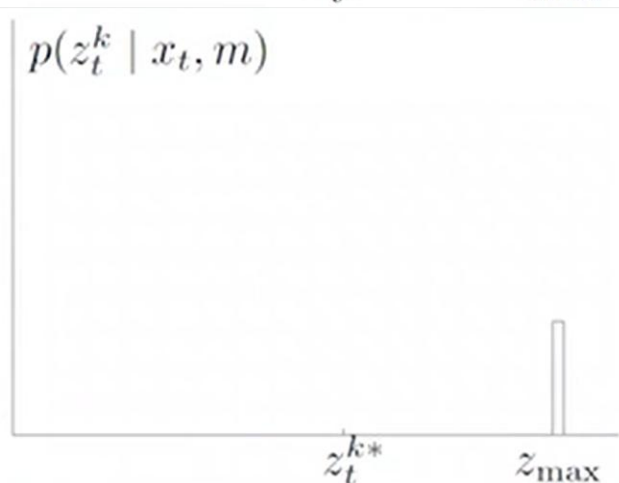
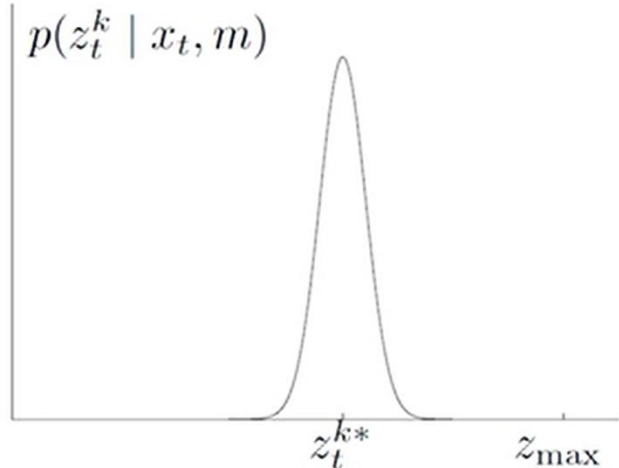
$$p_{hit}(z_t|x_t, m) = \begin{cases} N(z_t^{k*}, \sigma_{hit}^2) & 0 \leq z_t^k \leq z_{\max} \\ 0 & otherwise \end{cases}$$

■ 没有检测到障碍物 $p_{\max}(z_t|x_t, m)$

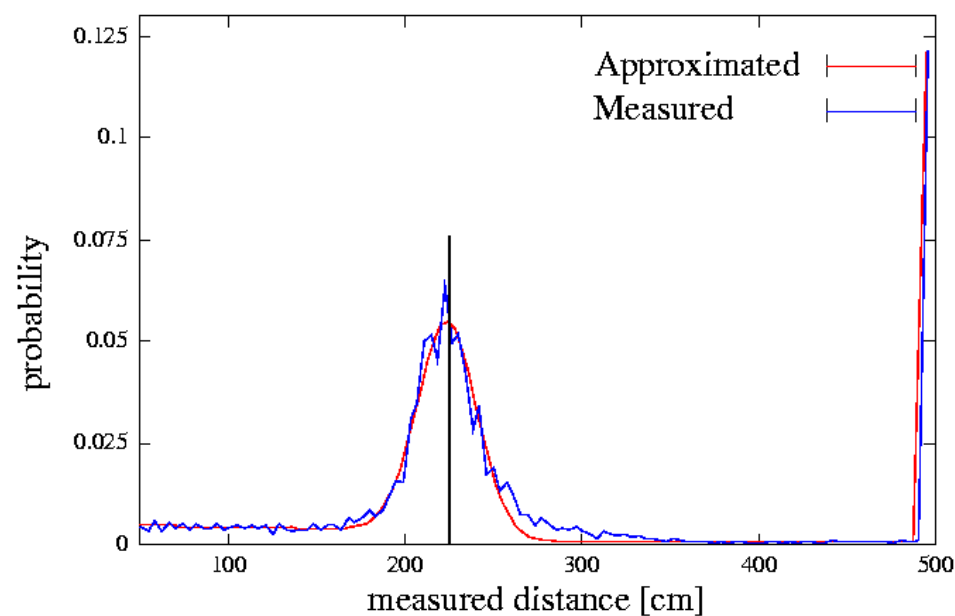
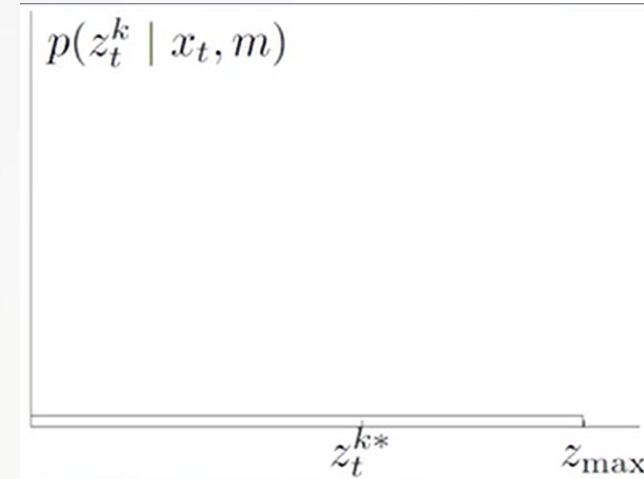
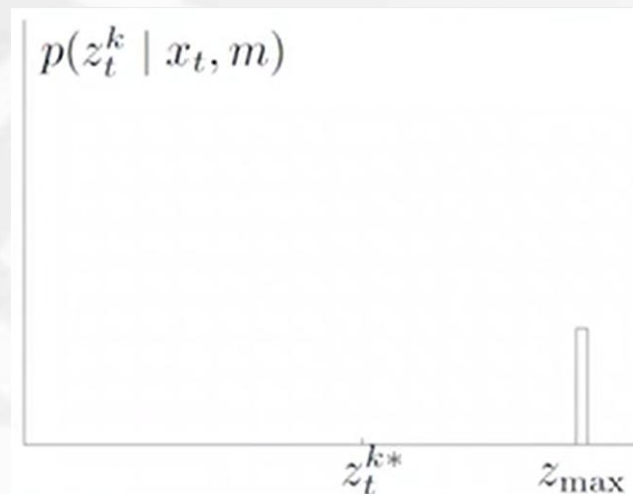
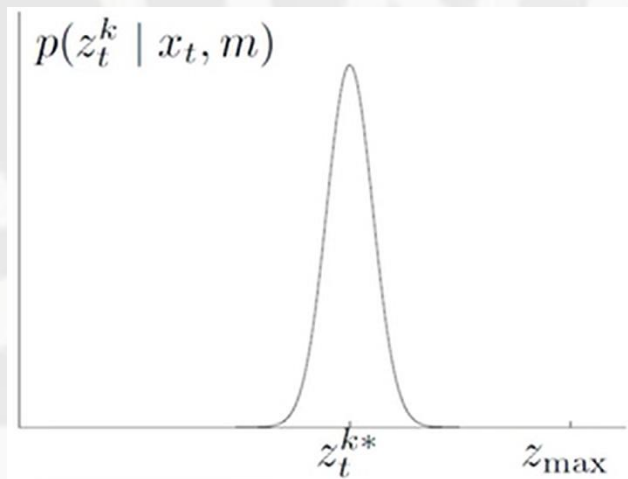
$$p_{\max}(z_t|x_t, m) = \begin{cases} 1 & z_t^k = z_{\max} \\ 0 & otherwise \end{cases}$$

■ 随机错误 $p_{rand}(z_t|x_t, m)$

$$p_{rand}(z_t|x_t, m) = \begin{cases} \frac{1}{z_{\max}} & 0 \leq z_t^k \leq z_{\max} \\ 0 & otherwise \end{cases}$$



2.8 激光雷达观测误差模型——物理模型



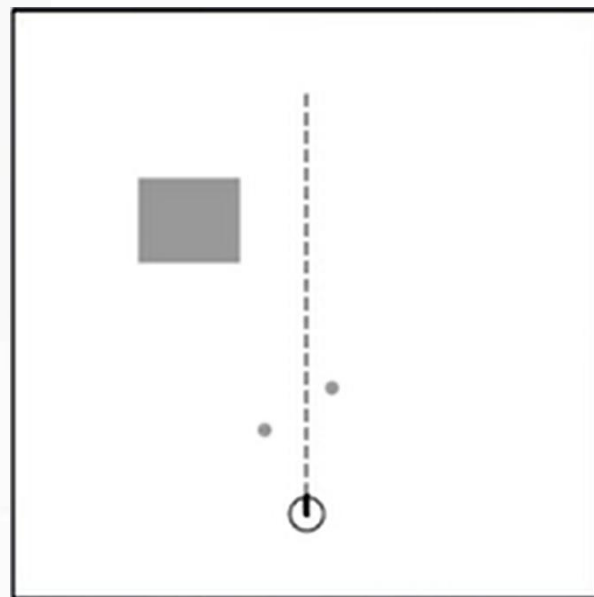
2.9 激光雷达观测误差模型——likelihood field

- 激光雷达传感器测量误差 $p_{hit}(z_t|x_t, m)$
 - ✓ 将激光束末端投影到地图中
 - ✓ 计算投影距离这个坐标距离最近的物体上的点，计算两者间的距离
 - ✓ 物体上距离最近的点作为均值，激光测距偏差作为方差
 - ✓ 可离线用可能域地图 (likelihood field) 表示
- 没有检测到障碍物 $p_{max}(z_t|x_t, m)$
- 随机错误 $p_{rand}(z_t|x_t, m)$
- 激光雷达观测误差模型 $p(z_t|x_t, m)$

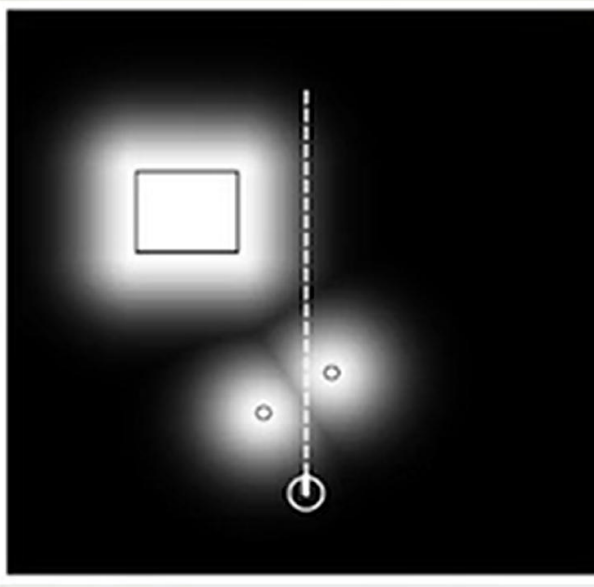
$$p(z_t|x_t, m) = \alpha_{hit}p_{hit} + \alpha_{max}p_{max} + \alpha_{rand}p_{rand}$$

$$\alpha_{hit} + \alpha_{max} + \alpha_{rand} = 1$$

障碍物地图

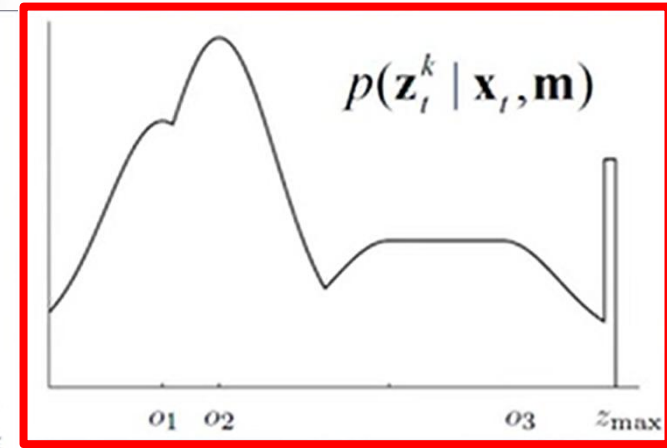
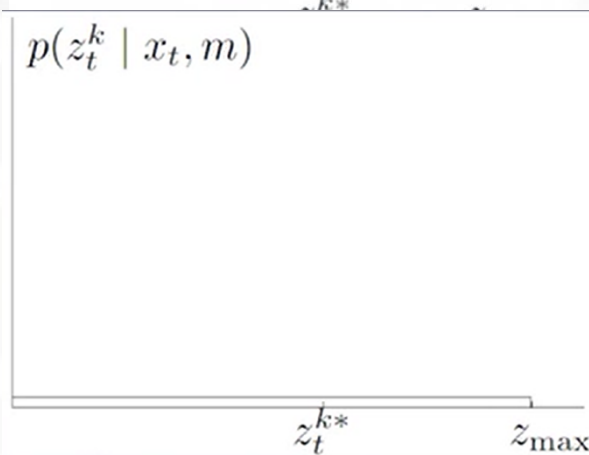
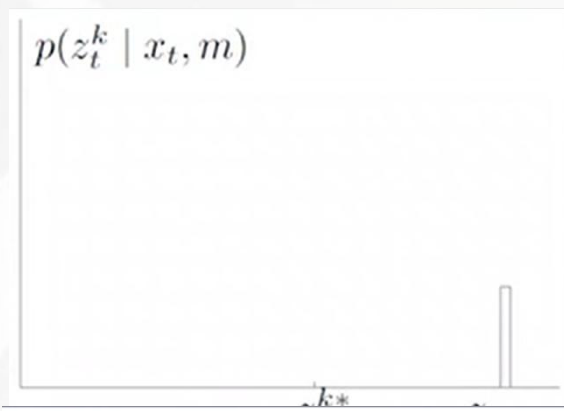
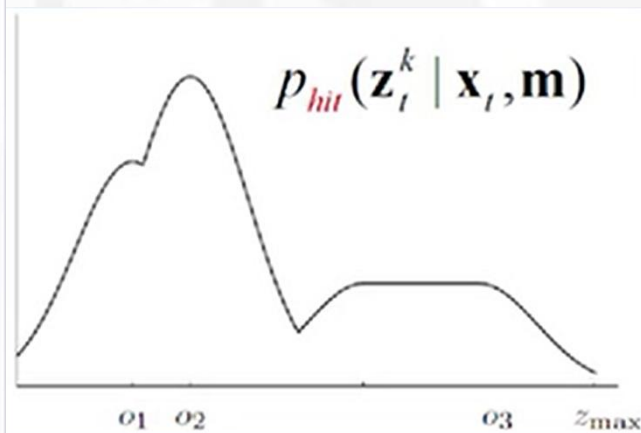
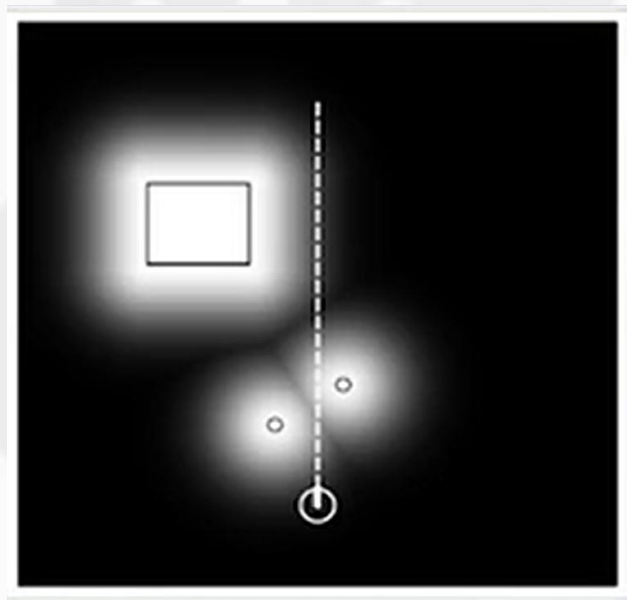


可能域地图



2.9 激光雷达观测模型——likelihood field

- 激光雷达传感器测量误差 $p_{hit}(z_t | x_t, m)$
 - ✓ 物体上距离最近的点作为均值，激光测距偏差作为方差
 - ✓ 可离线用可能域地图 (likelihood field) 表示
- 没有检测到障碍物 $p_{max}(z_t | x_t, m)$
- 随机错误 $p_{rand}(z_t | x_t, m)$



2.9 激光雷达观测模型——likelihood field

■ 优点:

- ✓ 在线计算量小
- ✓ 平滑性好, 可确保算法收敛
- ✓ 更加符合实际情况

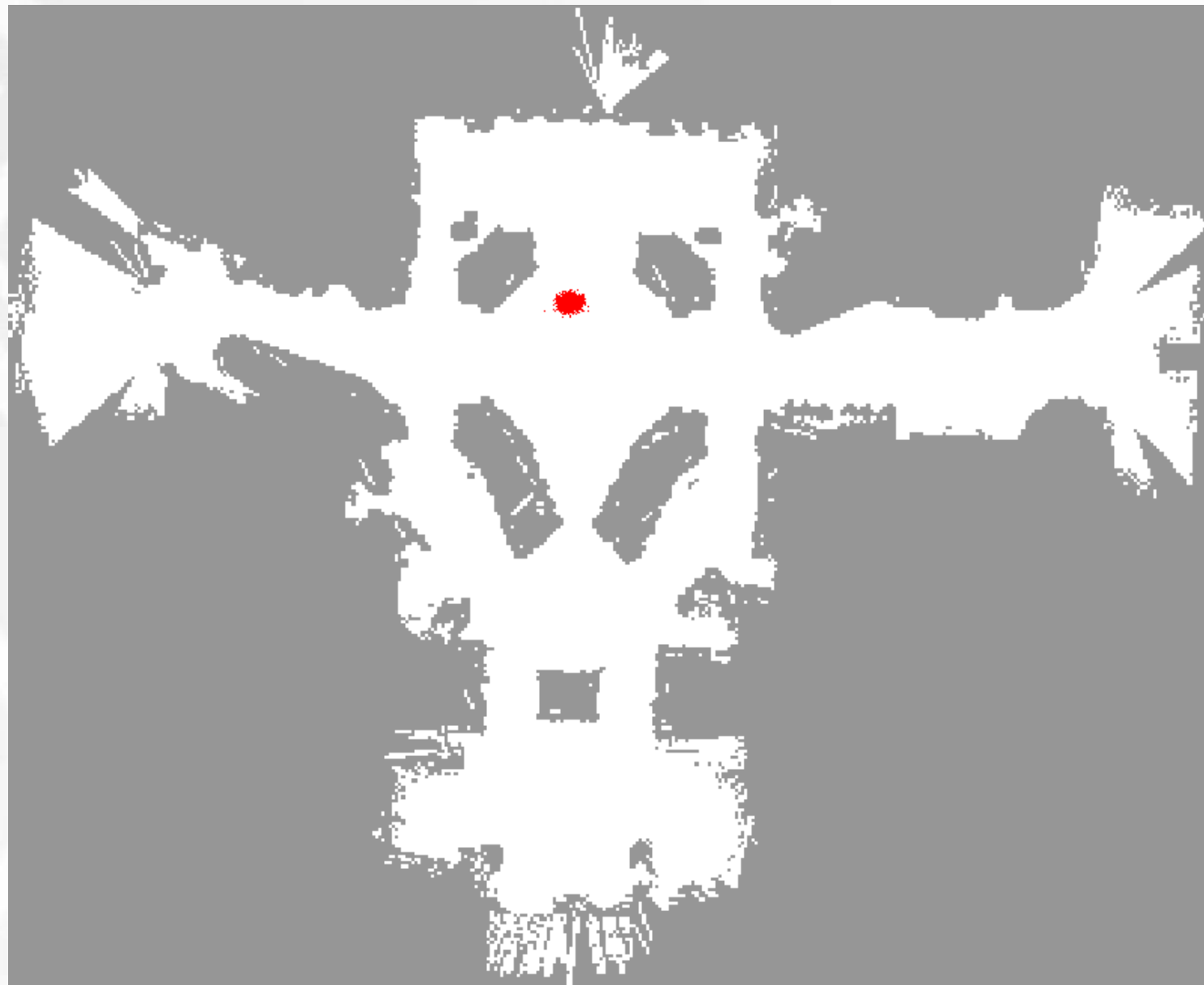
■ 缺点:

- ✓ 没有明确的物理意义
- ✓ 仅适用于相对静态的环境

```
1:  Algorithm likelihood_field_range_finder_model( $z_t, x_t, m$ ):  
2:       $q = 1$   
3:      for all  $k$  do  
4:          if  $z_t^k \neq z_{\max}$   
5:               $x_{z_t^k} = x + x_{k,\text{sens}} \cos \theta - y_{k,\text{sens}} \sin \theta + z_t^k \cos(\theta + \theta_{k,\text{sens}})$   
6:               $y_{z_t^k} = y + y_{k,\text{sens}} \cos \theta + x_{k,\text{sens}} \sin \theta + z_t^k \sin(\theta + \theta_{k,\text{sens}})$   
7:               $dist = \min_{x', y'} \left\{ \sqrt{(x_{z_t^k} - x')^2 + (y_{z_t^k} - y')^2} \mid \langle x', y' \rangle \text{ occupied in } m \right\}$   
8:               $q = q \cdot (z_{\text{hit}} \cdot \text{prob}(dist, \sigma_{\text{hit}}) + \frac{z_{\text{random}}}{z_{\max}})$   
9:      return  $q$ 
```

程序 6.3 利用与最近物体的欧氏距离计算测距仪似然的算法 [程序中函数 $\text{prob}(dist, \sigma_{\text{hit}})$ 计算在以 0 为中心标准方差为 σ_{hit} 的高斯分布的距离概率]

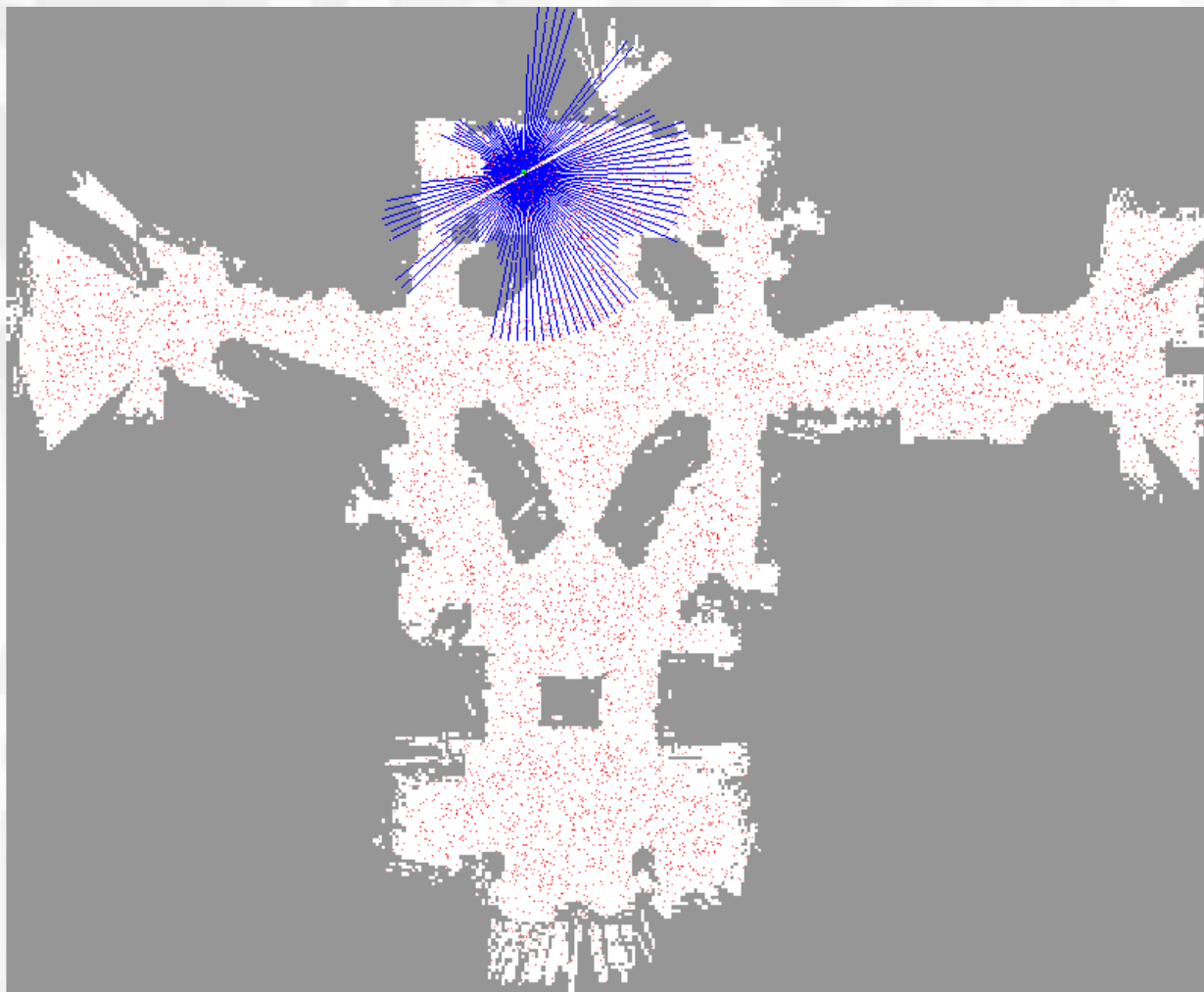
2.10 激光定位算法仿真实验



2.10 激光定位算法仿真实验

■ SVD匹配误差

$$\sum_{i=1}^n w_i \| (Rp_i + t) - q_i \|^2 = \sum_{i=1}^n w_i x_i^T x_i + \sum_{i=1}^n w_i y_i^T y_i - 2 \sum_{i=1}^n \sigma_i$$



2.11 蒙特卡洛（粒子滤波）算法总结

■ 优点:

- ✓ 可解决大范围全局定位问题
- ✓ 可以适用于多种分布模型
- ✓ 计算简单

■ 缺点:

- ✓ 当粒子数量过大的情况下，占用大量内存，计算效率会非常的低
 - 特别是一旦机器人确定了自己的位置之后只需要一小部分粒子就足以跟踪机器人的位置了
- ✓ 如果在非常大的地图环境下，少量的sample数又会导致定位的发散。
- ✓ 无法判断机器人绑架问题（定位错误问题）

03

自适应蒙特卡洛定位算法

AMCL: Augmented Monte Carlo Localization

3.1 自适应蒙特卡洛定位算法 (AMCL)

■ 自适应体现在：

- ✓ 解决了机器人绑架问题，它会在发现粒子们的平均分数突然降低了（意味着正确的粒子在某次迭代中被抛弃了）的时候，在全局再重新撒一些粒子。
- ✓ 解决了粒子数固定的问题，因为有时候当机器人定位差不多得到了的时候，比如这些粒子都集中在一块了，还要维持这么多的粒子没必要，这个时候粒子数可以少一点了。

■ 解决方法：

- ✓ 采用KLD(Kullback-Leibler Divergence)采样：库尔贝克-莱布勒散度，它是一种计算两个概率分布之间差异的方法
- ✓ KLD采样核心思想就是根据基于采样近似质量的统计界限来确定粒子数量。在粒子滤波器的每次迭代过程中，KLD采样以概率 $1 - \sigma$ 来确定样本数量，真实的后验概率与基于采样的近似分布之间的差异小于 ε 。对应于实际问题，就是在栅格地图中，看粒子占了多少栅格。占得多，说明粒子很分散，在每次迭代重采样的时候，允许粒子数量的上限高一些。占得少，说明粒子都已经集中了，那就将上限设低，采样到这个数就行了。

3.2 自适应蒙特卡洛定位算法流程

Algorithm MCL(X_{t-1}, u_t, z_t):

$\bar{X}_t = X_t = \emptyset$

for $m = 1$ to M :

$x_t^{[m]} = \text{motion_update}(u_t, x_{t-1}^{[m]})$

$w_t^{[m]} = \text{sensor_update}(z_t, x_t^{[m]})$

$\bar{X}_t = \bar{X}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$

endfor

for $m = 1$ to M :

draw $x_t^{[m]}$ from $\bar{X}_t$ with probability $\propto w_t^{[m]}$

$X_t = X_t + x_t^{[m]}$

endfor

return X_t

Algorithm Augmented_MCL($\mathcal{X}_{t-1}, u_t, z_t, m$):

static $w_{\text{slow}}, w_{\text{fast}}$

$\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$

for $m = 1$ to M do

$x_t^{[m]} = \text{sample_motion_model}(u_t, x_{t-1}^{[m]})$

$w_t^{[m]} = \text{measurement_model}(z_t, x_t^{[m]}, m)$

$\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$

$w_{\text{avg}} = w_{\text{avg}} + \frac{1}{M} w_t^{[m]}$

endfor

$w_{\text{slow}} = w_{\text{slow}} + \alpha_{\text{slow}}(w_{\text{avg}} - w_{\text{slow}})$

$w_{\text{fast}} = w_{\text{fast}} + \alpha_{\text{fast}}(w_{\text{avg}} - w_{\text{fast}})$

for $m = 1$ to M do

with probability $\max(0.0, 1.0 - w_{\text{fast}}/w_{\text{slow}})$ do
add random pose to $\mathcal{X}_t$

else

draw $i \in \{1, \dots, N\}$ with probability $\propto w_t^{[i]}$

add $x_t^{[i]}$ to $\mathcal{X}_t$

endwith

endfor

return $\mathcal{X}_t$

3.3 AMCL算法与MCL算法区别

- 长期指数滤波器衰减率 α_{slow}
- 长期重要性指数似然评价估计 w_{slow} :
$$w_{slow} = w_{slow} + \alpha_{slow}(\omega_{avg} - w_{slow})$$
- 短期指数滤波器衰减率 α_{fast}
- 短期重要性指数似然评价估计 w_{fast}
$$w_{fast} = w_{fast} + \alpha_{fast}(\omega_{avg} - w_{fast})$$
- $0 < \alpha_{slow} \ll \alpha_{fast}$: ROS中默认取0.001和0.1
- 当系统出现机器人绑架等问题时, 粒子的平均权重 w_{avg} 会开始下降, 导致 $w_{slow} > w_{fast}$, 算法将会按照 $\max(0, 1 - w_{fast}/w_{slow})$ 的概率往粒子集里面注入随机粒子
- 当粒子聚集(机器人获得了位姿信息后), $w_{slow} < w_{fast}$, 此时随机粒子不再添加, 而是从原先的粒子集中筛选权重较大的。

Algorithm Augmented_MCL($\mathcal{X}_{t-1}, u_t, z_t, m$):

```
static  $w_{slow}, w_{fast}$ 
 $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$ 
for  $m = 1$  to  $M$  do
     $x_t^{[m]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[m]})$ 
     $w_t^{[m]} = \text{measurement\_model}(z_t, x_t^{[m]}, m)$ 
     $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$ 
     $w_{avg} = w_{avg} + \frac{1}{M} w_t^{[m]}$ 
endfor
 $w_{slow} = w_{slow} + \alpha_{slow}(w_{avg} - w_{slow})$ 
 $w_{fast} = w_{fast} + \alpha_{fast}(w_{avg} - w_{fast})$ 
for  $m = 1$  to  $M$  do
    with probability  $\max(0.0, 1.0 - w_{fast}/w_{slow})$  do
        add random pose to  $\mathcal{X}_t$ 
    else
        draw  $i \in \{1, \dots, N\}$  with probability  $\propto w_t^{[i]}$ 
        add  $x_t^{[i]}$  to  $\mathcal{X}_t$ 
    endwith
endfor
return  $\mathcal{X}_t$ 
```




南开大学
Nankai University



天津市智能机器人技术重点实验室
Tianjin Key Laboratory of Intelligent Robotics
南开大学机器人与信息自动化研究所
Institute of Robotics & Automatic Information System

第五章 移动机器人感知技术5

蒙特卡洛定位算法(MCL)

孙雷 教授

Tel:13512967601

Email: sunl@nankai.edu.cn

2026年5月11日